

CoppeliaSim Tutorial 6

複数リンクモデルへの外力適用：2リンクアーム

Tutorial 5では単一の物体に外力を加えましたが、今回は**複数のパーツがジョイントで連結されたモデル**に外力を加える方法を学びます。これは人体の関節運動や、歩行器使用時の外乱応答など、より実践的なシミュレーションの基礎になります。

目的

- **複数リンク（マルチボディ）モデル** を作成する方法を理解する
 - 特定のリンクに**選択的に外力を加える**
 - 力が**関節を通じて伝播する**様子を観察する
 - 臨床的な外乱シミュレーションの基礎を習得する
-

使用環境

- CoppeliaSim（Bullet 物理エンジン）
 - **前提:** Tutorial 1～5 を完了していること
-

1. シーンの準備

今回作成するのは「床に固定された2リンクアーム」です。

[固定台] ---- (肩関節) ---- [上腕] ---- (肘関節) ---- [前腕]

1.1 新規シーンを作成

1. **File → New scene**

1.2 床について

新規シーンには既に **Floor** が存在します。デフォルトの **Floor** はすでに静的で衝突判定もあるので、そのまま使います。特に設定変更は不要です。

2. 固定台（BaseBlock）を作成

アームの土台となる部分を作ります。これは**静的オブジェクト**（動かない）です。

2.1 形状を追加

1. **Add → Primitive shape → Cuboid**
2. サイズ: X=0.15, Y=0.15, Z=0.6

3. ☐ **Dynamic and responsible** は **チェックしない**
4. 名前を **BaseBlock** に変更
5. Position: X=0, Y=0, Z=**0.3**
6. Orientation: Alpha=0, Beta=0, Gamma=0

💡 **サイズ変更について:** 作成済みのオブジェクト（プリミティブ）のサイズを変更するのは少し手順が複雑です。サイズを修正したい場合は、**そのオブジェクトを削除して、正しいサイズで作り直す**のが最も簡単で確実です。

2.2 色を変更

1. **Adjust color** → グレー（例: 0.5, 0.5, 0.5）

3. 設定一覧（クイックリファレンス）

以下の表に従って各オブジェクトを作成してください：

オブジェクト	サイズ	Position (X, Y, Z)	Orientation (α, β, γ)	Dynamic	質量
BaseBlock	0.15×0.15× 0.6	0, 0, 0.3	0, 0, 0	☐ NO	-
ShoulderJoint	-	0.1, 0, 0.6	90 , 0, 0	Dynamic/Free	-
UpperArm	0.05×0.05×0.2	0.1, 0, 0.5	0 , 0, 0	☒ YES	0.1kg
ElbowJoint	-	0.1, 0, 0.4	90 , 0, 0	Dynamic/Free	-
ForeArm	0.04×0.04×0.15	0.1, 0, 0.325	0 , 0, 0	☒ YES	0.05kg

💡 ポイント:

1. ジョイントは **Alpha=90** (回転軸を水平にするため)、アームは **Alpha=0** (垂直にぶら下げるため) に設定します。
2. アームが台座(BaseBlock)と衝突しないよう、X座標を **0.1** ずらして配置します。

4. 上腕（UpperArm）を作成

4.1 形状を追加

1. **Add** → **Primitive shape** → **Cuboid**
2. サイズ: X=0.05, Y=0.05, Z=0.2
3. ☒ **Create dynamic and responsible shape** にチェック
4. 名前を **UpperArm** に変更
5. Position: **X=0.1, Y=0, Z=0.5**
6. Orientation: **Alpha=0, Beta=0, Gamma=0**

4.2 質量を設定

1. **Dynamic properties dialog** → **Mass** を **0.1 kg** に設定

4.3 色を変更

1. **Adjust color** → 青色 (例: 0.2, 0.4, 0.8)
-

5. 肩関節 (ShoulderJoint) を作成

5.1 ジョイントを追加

1. **Add → Joint → Revolute**
2. 名前を **ShoulderJoint** に変更
3. Position: **X=0.1, Y=0, Z=0.6** (BaseBlockの脇)
4. Orientation: **Alpha=90, Beta=0, Gamma=0**

💡 Alpha=90 でY軸まわりに回転。アームが重力で垂れ下がります。

5.2 ジョイントのモードを設定

1. **Mode: Dynamic mode**
2. **Dynamic properties dialog → Control mode: Free**

5.3 階層を構築

```
BaseBlock
├── ShoulderJoint
│   └── UpperArm
```

6. 前腕 (ForeArm) を作成

6.1 形状を追加

1. **Add → Primitive shape → Cuboid**
2. サイズ: X=0.04, Y=0.04, Z=0.15
3. ☒ **Create dynamic and responsible shape** にチェック
4. 名前を **ForeArm** に変更
5. Position: **X=0.1, Y=0, Z=0.325**
6. Orientation: **Alpha=0, Beta=0, Gamma=0**

6.2 質量を設定

1. **Mass** を **0.05 kg** に設定

6.3 色を変更

1. **Adjust color** → 赤色 (例: 0.8, 0.2, 0.2)
-

7. 肘関節 (ElbowJoint) を作成

7.1 ジョイントを追加

1. **Add → Joint → Revolute**
2. 名前を **ElbowJoint** に変更
3. Position: **X=0.1, Y=0, Z=0.4** (UpperArmの下端)
4. Orientation: **Alpha=90, Beta=0, Gamma=0**

7.2 ジョイントのモードを設定

1. **Mode: Dynamic mode**
2. **Control mode: Free**

7.3 最終的な階層構造

```
Floor
BaseBlock
├── ShoulderJoint
│   └── UpperArm
│       └── ElbowJoint
│           └── ForeArm
```

8. シミュレーションテスト

8.1 重力テスト

1. ▶ **Start simulation**
2. 期待される動作:
 - アームは最初から垂直になっているため、**大きくは動きません**（これが正常です）。
 - **重要:** アームがバラバラに落下せず、**繋がったままぶら下がっている**ことを確認してください。
3.  **Stop simulation**

💡 うまくいかない場合（落下してしまう場合）：

- BaseBlockの **Dynamic** が無効（静的）になっているか確認
- Scene hierarchy で親子関係（階層構造）が正しいか確認

8. 外力を加えるスクリプト

前腕（ForeArm）に外力を加えて、アーム全体の応答を観察します。

8.1 スクリプトを追加

1. Scene hierarchy で **ForeArm** を右クリック
2. **Add → Script → Simulation script → Non-threaded → Lua**

8.2 スクリプトを編集

△ コピー&ペースト時の注意: 以下のコードには `function ... end` の塊が 2つ (`sysCall_init` と `sysCall_actuation`) あります。枠線の部分 (``lua や ``) は含めずに、これら2つの関数をすべてまとめて コピーして貼り付けてください。

```
function sysCall_init()
    -- 1. Get the object this script is attached to
    foreArmHandle = sim.getObject('.')

    -- 2. If not a Shape, check parent (..)
    if sim.getObjectType(foreArmHandle) ~= sim.object_shape_type then
        foreArmHandle = sim.getObject('..')
    end

    -- 3. If still not a Shape, search by name "ForeArm" (fallback)
    if sim.getObjectType(foreArmHandle) ~= sim.object_shape_type then
        foreArmHandle = sim.getObject('/ForeArm')
    end

    forceDelay = 1.5      -- Time to apply force (seconds)
    forceMagnitude = 2.0  -- Force magnitude (Newtons)
    forceApplied = false  -- Flag: force applied or not

    print("=== 2-Link Arm Force Demo Ready ===")
end

function sysCall_actuation()
    local t = sim.getSimulationTime()

    -- Apply force once after delay
    if not forceApplied and t >= forceDelay then
        -- Force direction: +X axis (push sideways)
        local force = {forceMagnitude, 0, 0}

        -- Force point: ForeArm tip (0.075m below center)
        local position = {0, 0, -0.075}

        -- Apply the force
        sim.addForce(foreArmHandle, position, force)

        forceApplied = true
        print("Force applied!")
    end
end
```

8.3 コードの解説

変数/関数	説明
<code>foreArmHandle</code>	ForeArmオブジェクトのハンドル
<code>forceMagnitude = 2.0</code>	2ニュートンの力（軽いアームなので小さめ）

変数/関数	説明
<code>force = {2.0, 0, 0}</code>	X軸プラス方向（横向き）に力を加える
<code>position = {0, 0, -0.075}</code>	ForeArmの先端（中心から下方向へ0.075m）

9. シミュレーション実行と観察

9.1 実行手順

1.  **Real-time mode** をONに
2. ▶ **Start simulation**
3. 1.5秒後に前腕の先端に力が加わります
4. **観察ポイント:**
 - 前腕だけでなく、**上腕も連動して動く**
 - **両方のジョイント（肩と肘）が回転する**
 - 揺れながら徐々に減衰していく

 **観察:** これが「多リンク系の力の伝播」です。末端に力を加えると、ジョイントを通じて全体に影響が波及します。

9.2 力とトルクの詳細

 **ポイント:** このスクリプトでは力は**1フレーム（瞬間的）**だけ加えられます。継続的な力ではなく、「軽く押す」イメージです。

前腕の先端にX方向の力を加えると、**両方のジョイントにトルクが発生**します：

ジョイント	モーメントアーム（力の作用点までの距離）	トルクの大きさ
肩関節	約0.35m（肩→前腕先端）	大きい
肘関節	約0.075m（肘→前腕先端）	小さい

肩関節へのトルクが肘の約5倍大きいため、**肩の動きが目立ち、肘の曲がり相対的に見えにくく**なります。セクション11の「継続的な力」を試すと、肘の曲がりもより観察しやすくなります。

10. 実験：異なるリンクに力を加える

実験1: 上腕だけに力を加える

前腕ではなく上腕に力を加えた場合の動きを観察してみましょう。

1. **ForeArmのスクリプトを無効化:**
 - Scene hierarchy で ForeArm の下にあるスクリプトアイコン () をダブルクリック
 - **Scene Object Properties** ダイアログが開く（後ろに隠れている場合があります）
 - **Script** タブの **"Enabled"** のチェックを外す
 - ダイアログを閉じる
2. **UpperArm** に新しいスクリプトを追加 (Add → Script → Simulation script → Non-threaded → Lua)
3. 以下のコードを使用：

```

function sysCall_init()
    -- Find UpperArm (self -> parent -> name search)
    upperArmHandle = sim.getObject('.')

    if sim.getObjectType(upperArmHandle) ~= sim.object_shape_type then
        upperArmHandle = sim.getObject('..')
    end

    if sim.getObjectType(upperArmHandle) ~= sim.object_shape_type then
        upperArmHandle = sim.getObject('/UpperArm')
    end

    forceDelay = 1.5
    forceApplied = false
    print("=== UpperArm Force Script Ready ===")
end

function sysCall_actuation()
    local t = sim.getSimulationTime()

    if not forceApplied and t >= forceDelay then
        local force = {3, 0, 0} -- Stronger force for heavier part
        local position = {0, 0, 0} -- Push at center

        sim.addForce(upperArmHandle, position, force)

        forceApplied = true
        print("Force applied to UpperArm!")
    end
end

```

💡 **比較ポイント:** 上腕に力を加えた場合と、前腕に力を加えた場合で、動きがどう違うか観察してください。

実験2: 複数のリンクに同時に力を加える

両方のスクリプトを有効にして、上腕と前腕に同時に力を加えてみましょう。

1. ForeArmのスクリプトを再度有効化:

- Scene hierarchy で ForeArm の下にあるスクリプトアイコン (📄) をダブルクリック
- **Scene Object Properties** ダイアログの **Script** タブで **"Enabled"** にチェックを入れる
- ダイアログを閉じる

2. ▶ Start simulation

3. 1.5秒後に両方のリンクに同時に力が加わる様子を観察

11. 応用：継続的な力を加える

Tutorial 5で学んだ継続的な力を、このマルチリンクモデルにも適用できます。

ForeArmのスクリプトを以下に置き換えてみましょう：

```
function sysCall_init()
    -- Find ForeArm (self -> parent -> name search)
    foreArmHandle = sim.getObject('.')

    if sim.getObjectType(foreArmHandle) ~= sim.object_shape_type then
        foreArmHandle = sim.getObject('..')
    end

    if sim.getObjectType(foreArmHandle) ~= sim.object_shape_type then
        foreArmHandle = sim.getObject('/ForeArm')
    end

    startTime = 1.0
    duration = 2.0

    print("=== Continuous Force Demo ===")
end

function sysCall_actuation()
    local t = sim.getSimulationTime()

    -- Apply force continuously during the specified period (1.0s - 3.0s)
    if t >= startTime and t < startTime + duration then
        local force = {0.5, 0, 0} -- Weak continuous force
        local position = {0, 0, -0.075} -- Push at tip

        sim.addForce(foreArmHandle, position, force)
    end
end
```

 **理学療法との関連:** これは「介助者が一定の力で押し続ける」状況をシミュレートしています。たとえば、立位で肩を一定時間押され続けた場合の姿勢応答などに応用できます。

11.1 なぜ肘が曲がらないのか？

継続的な力を加えると、アームが大きく回転してBaseBlockにぶつかります。しかし、**肘はほとんど曲がりません**。これは重要な物理的概念に関係しています。

ワールド座標系 vs ローカル座標系

```
local force = {0.5, 0, 0} -- This is WORLD coordinate system
```

この力は****ワールド座標系の固定方向 (+X) ****に加えられます。前腕がどの向きに回転しても、力の方向は変わりません。

座標系	力の方向	特徴
-----	------	----

座標系	力の方向	特徴
ワールド座標系	常に+X方向（固定）	アームの姿勢によっては肘を 伸展 させる方向に
ローカル座標系	前腕の向きに追従	常に肘を 屈曲 させる方向に

動きの解析

0度～90度：力(+X)は肩を回転させる → 加速
90度：力と回転軸が平行 → トルクほぼゼロ、しかし慣性で継続
90度～：力(+X)は逆方向に作用 → 減速するが、慣性で180度近くまで到達
最終：BaseBlockへの衝突で停止

肘が曲がらない理由

アームが大きく回転した後の姿勢では：

- 力（+X方向）は肘を**屈曲**させる方向ではなく
- 肘を**伸展**させる方向に作用している
- そのため肘はほとんど曲がらない

 **重要:** 実際の介助場面では、介助者の手は患者の動きに**追従**します（ローカル座標系的）。固定方向に力を加え続けることは稀です。この違いを理解することは、シミュレーションと実際の臨床場面を結びつける上で重要です。

12. シーンを保存

- File → Save scene as...
- ファイル名: two_link_arm.ttt

 トラブルシューティング

症状	チェック項目
アームが落下してしまう	BaseBlockは Dynamic を無効にしていますか？
ジョイントが動かない	ジョイントの Mode が Dynamic になっていますか？
階層が正しくない	Scene hierarchy で親子関係を確認してください
力が効かない	オブジェクトの Dynamic が有効か確認してください
動きが激しすぎる	質量(Mass)を増やすか、力(forceMagnitude)を減らしてください
すぐに止まってしまう	ジョイントの Damping が大きすぎる可能性があります

 物理的な補足

多リンク系の特徴

概念	説明	臨床例
連鎖反応	1箇所に力を加えると、全リンクに影響	転倒時の連鎖的な関節運動
モーメントアーム	作用点が回転軸から遠いほど、大きな回転力	リーチ動作時の肩への負荷
慣性モーメント	末端ほど振り回されやすい	歩行時の下肢スイング

位置計算のコツ

各リンクの配置位置を計算するには：

- 各リンクの**中心位置**を考える
- ジョイントは**リンクの接続点（境界）**に配置

```
例：BaseBlock(高さ0.6, 中心Z=0.3)
  → 上端 Z=0.6
  → ShoulderJoint Z=0.6
  → UpperArm(高さ0.2) 中心 Z=0.5 (0.6 - 0.2/2: 下にぶら下げるのでマイナス方向)
  → 下端 Z=0.4
  → ElbowJoint Z=0.4
  → ForeArm(高さ0.15) 中心 Z=0.325 (0.4 - 0.15/2)
```

 次へのステップ

これで「複数リンクモデルへの外力適用」の基礎ができました！

次のチュートリアルでは：

- 転倒を検知するセンサー**を追加する
- 力の大きさをリアルタイムに変化させる UI**を作る
- より複雑な**人体モデル**へと発展させる

などに挑戦していきます。

 復習問題

- 多リンクモデルで、末端に力を加えると、他のリンクはどうなりますか？
- ジョイントの位置を決めるときに注意すべきことは何ですか？
- BaseBlockを **Dynamic** にするとどうなりますか？なぜそうなりますか？
- 上腕に力を加えた場合と、前腕に力を加えた場合で、動きはどう違いますか？